

PROGRAM

LEX:

```
%{
#include "y.tab.h"
%}
%%
[0-9]+          { return NUMBER; }
[_a-zA-Z][_a-zA-Z0-9]* { return ID; }
[ \t]           { /* ignore whitespace */ }
\n             { return 0; }
.               { return yytext[0]; }
```

```
%%
int yywrap() {
    return 1;
}
```

YACC:

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
int yyerror(const char *s);
%}
%token NUMBER ID
%left '+' '-'
%left '*' '/'
%%
exp : exp '+' exp
    | exp '-' exp
    | exp '*' exp
    | exp '/' exp
    | '(' exp ')'
    | NUMBER
    | ID
    ;

%%
int main()
{
    printf("Enter the expression: ");
    if (yyparse() == 0) {
        printf("Valid Expression!\n");
    }
    return 0;
}

int yyerror(const char *s) {
    printf("Invalid Expression!\n");
    exit(1);
}
```



Valid Arithmetic Expression using LEX and YACC

Aim: To program to recognize a valid arithmetic expression that uses operators $+$, $-$, $*$ and $/$.

Algorithm:

- 1) Start
- 2) write regular expression for identifier and number
- 3) write syntax for all possible arithmetic expression
- 4) Input the expression to be checked.
- 5) Output whether the expression is valid or not.
- 6) Stop.

Result: The program has been executed and output verified successfully.

PROGRAM

LEX:

```
%{
#include "y.tab.h"
%}
%%
[0-9] { return DIGIT; }
[a-zA-Z] { return ALPHA; }
\n { return 0; }
. { return yytext[0]; }
%%
```

YACC:

```
%{
#include <stdio.h>
#include <stdlib.h>
%}
%token DIGIT ALPHA
%%
var : ALPHA
| var ALPHA
| var DIGIT ;
%%
int main(int argc, char *argv[]) {
printf("Enter a variable name: ");
yyparse();
printf("Valid Variable!\n");
return 0;
}
int yyerror() {
printf("Invalid Variable!\n");
exit(1);
}
int yywrap() {
return 1;
}
```

OUTPUT

```
sreelakshmi@L5SYS20:~/cs2$ ./a.out
Enter a variable name: 2563agg
Invalid Variable!
sreelakshmi@L5SYS20:~/cs2$ ./a.out
Enter a variable name: abcd
Valid Variable!
```



Valid Identifier using LEX AND YACC

Aim: To recognize a valid identifier which starts with a letter followed by any number of letters or digits using lex and yacc.

Algorithm:

- 1) start
- 2) write the regular expression for identifier check.
- 3) Input the variable
- 4) Stop.

Result: The program has been executed and output verified successfully.

PROGRAM

LEX:

```
%{
#include "y.tab.h"
#include <math.h>
%}
%%
([0-9]+|([0-9]*\.[0-9]+)([eE][+-]?[0-9]+)?) { yylval.dval = atof(yytext); return NUMBER; }
log | LOG      { return LOG; }
ln | LN        { return nLOG; }
sin | SIN       { return SINE; }
cos | COS       { return COS; }
tan | TAN       { return TAN; }
mem            { return MEM; }
[\t]           ;
\$             return 0;
\n|.          return yytext[0];
%%
```

YACC:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
double memvar;
%}
%union
{
    double dval;
}
%token <dval> NUMBER
%token <dval> MEM
%token LOG SINE nLOG COS TAN
%left '-' '+'
%left '*' '/'
%right '^'
%left LOG SINE nLOG COS TAN
%nonassoc UMINUS
%type <dval> expression
%%
start
    : statement '\n'
    | start statement '\n'
    ;
statement
    : MEM '=' expression { memvar = $3; }
    | expression { printf("Answer = %g\n", $1); }
    ;
expression
    : expression '+' expression { $$ = $1 + $3; }
    | expression '-' expression { $$ = $1 - $3; }
```



Implementation of Calculator using LEX and YACC

Aim: To implementation of calculator using lex and yacc

Algorithm:

- 1) start
- 2) write function for each of the operator
- 3) Input the given expression
- 4) Perform lexical analyser
- 5) print output
- 6) stop.

Result: The program has been executed and output verified successfully.